

Project: Concorrenza economica

nucleo deterministico, rumore, eterogeneita' e apprendimento
adattivo della competizione strategica

Concorrenza economica: Cournot e Hotelling

Dispensa sul nucleo deterministico della competizione strategica

1. Obiettivi della dispensa

Questa dispensa introduce due modelli classici della teoria della concorrenza:

1. il modello di Cournot, in cui le imprese competono scegliendo quantita';
2. il modello di Hotelling, in cui le imprese competono scegliendo localizzazione e, nella versione piu' ricca, anche prezzo.

Gli obiettivi sono quattro:

1. comprendere la logica della competizione strategica in mercati oligopolistici;
2. derivare le funzioni di reazione e gli equilibri di Nash;
3. discutere come il numero di imprese e la localizzazione influenzino prezzi, quantita' e profitti;
4. costruire una base teorica che possa poi essere estesa a versioni con rumore, eterogeneita' e apprendimento adattivo.

Questa prima dispensa resta volutamente deterministica. La seconda introdurrà invece gli ingredienti stocastici e computazionali.

2. Idea generale

In molti problemi di concorrenza economica, il profitto di ciascun attore non dipende solo dalla sua scelta, ma anche da quella degli altri. Questo e' il cuore della teoria dei giochi applicata all'economia industriale.

Nel modello di Cournot, le imprese producono un bene omogeneo e scelgono quanta' produrre. Il prezzo di mercato si forma a partire dalla quantita' totale offerta.

Nel modello di Hotelling, invece, la competizione riguarda la posizione nello spazio dei prodotti o nello spazio geografico. Gli agenti non competono solo su “quanto produrre”, ma anche su “dove collocarsi” o “quanto differenziarsi”.

Entrambi i modelli servono a formalizzare una tensione di fondo:

- avvicinarsi agli altri può aumentare la domanda o la quota di mercato;
- differenziarsi dagli altri può attenuare la concorrenza diretta.

3. Il modello di Cournot

3.1 Struttura del problema

Consideriamo n imprese che producono un bene omogeneo. Indichiamo con

$$D_i \geq 0$$

la quantità prodotta dall'impresa i e con

$$D = \sum_{i=1}^n D_i$$

la produzione totale.

Il prezzo di mercato dipende dalla quantità totale. Seguiamo le note e scriviamo

$$p = f(D),$$

dove

$$f'(D) < 0.$$

Questa è l'inversa della curva di domanda: all'aumentare dell'offerta complessiva, il prezzo di mercato diminuisce.

3.2 Ricavo dell'impresa

Il ricavo dell'impresa i è

$$R_i = pD_i = D_i f(D).$$

Poiché D dipende anche dalle scelte degli altri, ogni impresa massimizza il proprio ricavo tenendo conto della produzione altrui.

3.3 Condizione del primo ordine

Derivando rispetto a D_i si ottiene

$$\frac{\partial R_i}{\partial D_i} = f(D) + D_i f'(D).$$

La condizione del primo ordine e' quindi

$$f(D) + D_i f'(D) = 0.$$

Questa e' la funzione di reazione implicita dell'impresa i . Essa dice che l'impresa sceglie la propria quantita' fino al punto in cui il beneficio marginale e il costo strategico di espandere la produzione si compensano.

3.4 Caso simmetrico

Nel caso simmetrico, tutte le imprese scelgono la stessa quantita':

$$D_1 = D_2 = \dots = D_n.$$

Se chiamiamo D la quantita' totale, allora ogni impresa produce

$$D_i = \frac{D}{n}.$$

Sostituendo nella condizione del primo ordine otteniamo

$$f(D) + \frac{D}{n} f'(D) = 0.$$

Moltiplicando per n :

$$nf(D) + Df'(D) = 0.$$

Poiche' $p = f(D)$, si puo' riscrivere anche come

$$np + D \frac{dp}{dD} = 0.$$

Questa e' la condizione di equilibrio simmetrico per un oligopolio con n imprese. E' anche una delle conclusioni centrali delle note: al crescere del numero di imprese, il prezzo di equilibrio si riduce.

3.5 Interpretazione economica

La relazione

$$np + D \frac{dp}{dD} = 0$$

mostra che l'equilibrio dipende dalla pendenza della domanda inversa e dal numero di imprese.

Intuitivamente:

- se il numero di imprese cresce, ciascuna tiene meno conto dell'effetto della propria produzione sul prezzo;
- di conseguenza la quantità totale tende a crescere;
- il prezzo di equilibrio tende a scendere.

Questo formalizza l'idea che la concorrenza abbassa i prezzi.

3.6 Confronto con monopolio

Nel caso di monopolio, $n = 1$, e la condizione diventa

$$p + D \frac{dp}{dD} = 0.$$

Nel caso di oligopolio simmetrico con $n > 1$, la condizione è

$$np + D \frac{dp}{dD} = 0.$$

Poiché il termine np cresce con n , l'intersezione con la curva del ricavo marginale si sposta verso prezzi più bassi. Questa è proprio l'idea richiamata nelle note con il confronto tra monopolio e oligopolio.

3.7 Esempio lineare

Supponiamo una domanda inversa lineare:

$$p = a - bD, \quad a > 0, \quad b > 0.$$

Allora

$$\frac{dp}{dD} = -b.$$

La condizione di equilibrio simmetrico diventa

$$n(a - bD) - bD = 0.$$

Quindi

$$na - (n + 1)bD = 0,$$

da cui

$$D^* = \frac{na}{(n + 1)b}.$$

La quantita' di ciascuna impresa e'

$$D_i^* = \frac{a}{(n + 1)b}.$$

Il prezzo di equilibrio e'

$$p^* = a - bD^* = a - \frac{na}{n + 1} = \frac{a}{n + 1}.$$

Questa formula rende molto chiaro il ruolo di n : al crescere del numero di imprese, il prezzo scende.

3.8 Equilibrio di Nash e funzioni di reazione

Il modello di Cournot e' un gioco statico a informazione completa. L'equilibrio che si cerca e' un equilibrio di Nash, cioe' una configurazione in cui nessuna impresa ha incentivo a modificare unilateralmente la propria quantita'.

Nel caso lineare con due imprese, per esempio, il profitto dell'impresa 1 e'

$$\pi_1 = D_1(a - b(D_1 + D_2)).$$

Derivando rispetto a D_1 otteniamo

$$\frac{\partial \pi_1}{\partial D_1} = a - 2bD_1 - bD_2.$$

La funzione di reazione dell'impresa 1 e' quindi

$$D_1 = \frac{a - bD_2}{2b}.$$

Analogamente,

$$D_2 = \frac{a - bD_1}{2b}.$$

L'intersezione delle due rette di reazione determina l'equilibrio di Nash.

Nel caso simmetrico si ottiene

$$D_1^* = D_2^* = \frac{a}{3b}.$$

Questa e' una versione elementare ma molto utile del problema.

4. Il modello di Hotelling: localizzazione semplice

4.1 Idea del modello

Passiamo ora alla competizione spaziale. Nella versione piu' semplice, immaginiamo consumatori uniformemente distribuiti su un intervallo, per esempio

$$\left[-\frac{1}{2}, \frac{1}{2} \right].$$

Due imprese scelgono una posizione, che indichiamo con

$$x_1, \quad x_2.$$

I consumatori acquistano dal negozio piu' vicino. Le note formulano proprio questa regola: il cliente sceglie lo shop piu' vicino.

4.2 Domanda e quota di mercato

Supponiamo

$$x_1 < x_2.$$

Il consumatore indifferente e' il punto medio tra le due posizioni:

$$\bar{x} = \frac{x_1 + x_2}{2}.$$

Se la densita' dei consumatori e' costante e pari a ρ , allora il numero di clienti della prima impresa e' proporzionale alla distanza tra l'estremo sinistro e il punto medio, mentre quello della seconda e' proporzionale alla distanza tra il punto medio e l'estremo destro.

Seguendo la forma delle note, i profitti possono essere scritti come

$$U_1 = p \left(\frac{x_1 + x_2}{2} + \frac{1}{2} \right),$$

$$U_2 = p \left(\frac{1}{2} - \frac{x_1 + x_2}{2} \right),$$

dove p rappresenta il prezzo unitario fissato in modo comune o assunto costante.

4.3 Incentivo a muoversi verso il centro

Derivando rispetto alle posizioni, si vede che:

$$\frac{\partial U_1}{\partial x_1} > 0,$$

$$\frac{\partial U_2}{\partial x_2} < 0.$$

Quindi:

- l'impresa 1 ha incentivo a spostarsi verso destra;
- l'impresa 2 ha incentivo a spostarsi verso sinistra.

Entrambe tendono quindi verso il centro. Questo e' il cuore della "legge di Hotelling": in molti mercati i concorrenti tendono a rendere i prodotti il piu' simili possibile, o comunque a collocarsi vicini nello spazio delle preferenze.

4.4 Equilibrio al centro

Nel modello semplice, l'esito della dinamica strategica e'

$$x_1 = x_2 = 0.$$

Cioe' entrambe le imprese convergono verso il centro del mercato. Le note sottolineano proprio questa tendenza alla minima differenziazione, e ne richiamano anche l'analogia con la politica, dove in un sistema bipartitico le piattaforme tendono a posizionarsi vicino al votante mediano.

4.5 Interpretazione

Questo risultato non va letto come una legge universale, ma come l'esito di ipotesi molto specifiche:

- consumatori distribuiti uniformemente;
- scelta del negozio piu' vicino;
- prezzo fissato o non strategico;

- assenza di costi aggiuntivi di differenziazione.

Tuttavia e' un risultato molto importante, perche' mostra come la competizione possa spingere alla convergenza delle scelte.

5. Il modello di Hotelling originale con prezzi

5.1 Struttura del problema

Consideriamo ora la versione piu' ricca presente nelle note. Due imprese, A e B , si trovano lungo un segmento di lunghezza ℓ .

L'impresa A si trova a distanza a dall'estremo sinistro, mentre l'impresa B si trova a distanza b dall'estremo destro. Tra le due imprese c'e' una distanza intermedia descritta dalle variabili x e y , con vincolo geometrico

$$a + x + y + b = \ell.$$

I prezzi sono

$$P_1, \quad P_2,$$

e il costo di trasporto per unita' di distanza e'

$$c.$$

I consumatori tengono conto sia del prezzo sia del costo di trasporto.

5.2 Consumatore indifferente

Il consumatore situato nel punto di separazione tra i due mercati e' indifferente tra acquistare da A e acquistare da B . La condizione e'

$$P_1 + cx = P_2 + cy.$$

Da qui si ricavano

$$x = \frac{1}{2}(\ell - a - b) + \frac{1}{2} \frac{P_2 - P_1}{c},$$

$$y = \frac{1}{2}(\ell - a - b) - \frac{1}{2} \frac{P_2 - P_1}{c}.$$

Queste sono le quantita' di mercato "intermedie" attribuite alle due imprese. Sono esattamente le espressioni annotate nella nota.

5.3 Quantita' vendute

La quantita' totale venduta dalla prima impresa e'

$$q_1 = a + x,$$

mentre per la seconda impresa

$$q_2 = b + y.$$

Sostituendo le espressioni di x e y otteniamo

$$q_1 = \frac{1}{2}(\ell + a - b) + \frac{1}{2} \frac{P_2 - P_1}{c},$$

$$q_2 = \frac{1}{2}(\ell - a + b) - \frac{1}{2} \frac{P_2 - P_1}{c}.$$

5.4 Profitti

I profitti sono

$$\pi_1 = P_1 q_1, \quad \pi_2 = P_2 q_2.$$

Quindi

$$\pi_1 = P_1 \left[\frac{1}{2}(\ell + a - b) + \frac{1}{2} \frac{P_2 - P_1}{c} \right],$$

$$\pi_2 = P_2 \left[\frac{1}{2}(\ell - a + b) - \frac{1}{2} \frac{P_2 - P_1}{c} \right].$$

5.5 Condizioni del primo ordine

Derivando rispetto ai prezzi si ottiene

$$\frac{\partial \pi_1}{\partial P_1} = \frac{\ell + a - b}{2} - \frac{P_1}{c} + \frac{P_2 - P_1}{2c},$$

$$\frac{\partial \pi_2}{\partial P_2} = \frac{\ell - a + b}{2} - \frac{P_2}{c} + \frac{P_1 - P_2}{2c}.$$

Imponendo le condizioni del primo ordine uguali a zero si ottengono due rette di reazione nei prezzi.

5.6 Equilibrio dei prezzi

Risolvendo il sistema si ottiene l'equilibrio:

$$P_1^* = c\ell + \frac{c}{3}(a - b),$$

$$P_2^* = c\ell - \frac{c}{3}(a - b).$$

Queste formule compaiono chiaramente nella pagina finale delle note. Esse mostrano che il prezzo di equilibrio dipende non solo dalla lunghezza del mercato e dal costo di trasporto, ma anche dall'asimmetria di localizzazione tra le due imprese.

5.7 Quantita' di equilibrio

Sostituendo i prezzi di equilibrio nelle espressioni di domanda si ottiene

$$q_1^* = \frac{1}{2} \left(\ell + \frac{a - b}{3} \right),$$

$$q_2^* = \frac{1}{2} \left(\ell - \frac{a - b}{3} \right).$$

Se le due imprese sono simmetriche, cioè se

$$a = b,$$

allora

$$P_1^* = P_2^* = c\ell,$$

e inoltre

$$q_1^* = q_2^* = \frac{\ell}{2}.$$

5.8 Interpretazione

Queste formule contengono un messaggio molto chiaro:

- se un'impresa è localizzata in modo più favorevole, può sostenere un prezzo più alto;
- la differenziazione spaziale riduce l'intensità della concorrenza di prezzo;
- la struttura del mercato dipende congiuntamente da posizione e pricing.

La concorrenza, quindi, non e' soltanto "quanto produrre" o "quanto far pagare", ma anche "dove collocarsi".

6. Cournot e Hotelling a confronto

I due modelli possono essere letti come due modi diversi di formalizzare la competizione strategica.

Cournot

- variabile strategica: quantita';
- interazione: il prezzo dipende dall'offerta totale;
- oggetto centrale: funzione di reazione nelle quantita'.

Hotelling

- variabile strategica: localizzazione, e nella versione ricca anche prezzo;
- interazione: i clienti scelgono in base a distanza e prezzo;
- oggetto centrale: spartizione del mercato e differenziazione.

Dal punto di vista didattico, il confronto e' molto utile perche' mostra come lo stesso linguaggio di equilibrio strategico possa essere applicato a problemi economici molto diversi.

7. Perche' questa prima dispensa e' importante

Questa dispensa non e' ancora una case study "stocastica" nel senso forte del corso. Tuttavia e' indispensabile per tre ragioni.

Primo, introduce il lessico di base:

- equilibrio di Nash;
- funzione di reazione;
- stabilita';
- dipendenza strategica.

Secondo, chiarisce due archetipi fondamentali della concorrenza:

- competizione in quantita';
- competizione in spazio o differenziazione.

Terzo, prepara il terreno per la seconda dispensa, in cui questi stessi modelli verranno modificati introducendo:

- domanda rumorosa;
- eterogeneita' degli agenti;
- apprendimento adattivo;
- bounded rationality;
- simulazioni Monte Carlo.

8. Conclusione

Il modello di Cournot e il modello di Hotelling rappresentano due pilastri della teoria della concorrenza. Nel primo, la variabile strategica è la quantità; nel secondo, la localizzazione e la differenziazione. In entrambi i casi, la scelta ottimale di ciascun attore dipende dalle scelte degli altri, e l'equilibrio emerge come punto di intersezione di comportamenti strategici reciprocamente compatibili.

Per un corso di metodi computazionali, questa prima dispensa ha il compito di fissare la struttura deterministica dei problemi. Solo dopo aver chiarito il modello classico ha senso introdurre rumore, eterogeneità e dinamiche adattive.

9. Bibliografia minima

1. Cournot, A. A. (1838). *Recherches sur les principes mathématiques de la théorie des richesses*.
2. Hotelling, H. (1929). *Stability in Competition*. *Economic Journal*, 39(153), 41-57.
3. Tirole, J. (1988). *The Theory of Industrial Organization*. MIT Press.
4. Shy, O. (1995). *Industrial Organization: Theory and Applications*. MIT Press.
5. Mas-Colell, A., Whinston, M. D., and Green, J. R. (1995). *Microeconomic Theory*. Oxford University Press.

Concorrenza economica con rumore, eterogeneità e apprendimento adattivo

Dispensa su modelli stocastici della competizione strategica

1. Obiettivi della dispensa

Questa seconda dispensa estende i modelli classici di Cournot e Hotelling introducendo tre ingredienti che li rendono più adatti a un corso di metodi computazionali per modelli stocastici:

1. rumore, cioè shock casuali su domanda, profitti o decisioni;
2. eterogeneità, cioè differenze strutturali tra imprese o tra consumatori;
3. apprendimento adattivo, cioè aggiustamento progressivo delle scelte invece di ottimizzazione istantanea perfetta.

Gli obiettivi sono cinque:

1. mostrare come un modello deterministico possa essere trasformato in un modello stocastico;
2. distinguere tra equilibrio puntuale e distribuzione degli esiti;
3. introdurre dinamiche adattive semplici e simulabili;
4. discutere stabilità, volatilità e dipendenza dalle condizioni iniziali;

5. fornire una base per laboratori con simulazione numerica e Monte Carlo.

La logica generale e' semplice: i modelli classici di Cournot e Hotelling non vengono abbandonati, ma usati come scheletro teorico su cui innestare rumore e apprendimento.

2. Perche' serve una seconda dispensa

Nella prima dispensa, Cournot e Hotelling erano trattati nella loro forma classica:

- imprese perfettamente razionali;
- informazione completa;
- scelta ottima istantanea;
- equilibrio deterministico.

Questa struttura e' importante, ma spesso troppo rigida per descrivere mercati reali. Nella pratica:

- la domanda non e' perfettamente prevedibile;
- i costi possono essere diversi;
- i consumatori non reagiscono tutti allo stesso modo;
- le imprese non calcolano sempre la best response esatta;
- le decisioni possono essere adattive, imitate o rumorose.

Per questo motivo, il passaggio dal modello classico al modello stocastico non e' un dettaglio tecnico, ma un cambiamento concettuale.

3. Cournot con domanda rumorosa

3.1 Struttura del modello

Consideriamo due imprese che scelgono quantita'

$$q_1(t), \quad q_2(t).$$

La produzione totale e'

$$Q(t) = q_1(t) + q_2(t).$$

Nel modello classico con domanda lineare il prezzo era

$$p(t) = a - bQ(t).$$

Introduciamo ora un termine casuale di domanda:

$$p(t) = a - bQ(t) + \varepsilon_t,$$

dove

$$\varepsilon_t$$

e' uno shock casuale a media nulla.

L'interpretazione e' immediata: in ogni periodo il mercato puo' essere temporaneamente piu' favorevole o meno favorevole del previsto.

3.2 Profitto stocastico

Il profitto dell'impresa 1 diventa

$$\pi_1(t) = q_1(t)(a - b(q_1(t) + q_2(t)) + \varepsilon_t).$$

Analogamente,

$$\pi_2(t) = q_2(t)(a - b(q_1(t) + q_2(t)) + \varepsilon_t).$$

A questo punto il profitto non e' piu' una quantita' deterministica, ma una variabile casuale.

3.3 Due modi di leggere il problema

Questa semplice modifica apre due interpretazioni diverse.

Versione A – equilibrio in media

Le imprese massimizzano il profitto atteso:

$$\mathbb{E}[\pi_i].$$

Se

$$\mathbb{E}[\varepsilon_t] = 0,$$

allora l'equilibrio medio resta formalmente uguale a quello classico. Tuttavia, i profitti realizzati oscillano intorno a quel valore medio.

Versione B – decisione sotto incertezza

Le imprese non conoscono lo shock futuro e devono decidere in presenza di rischio. In questo caso non conta solo il profitto medio, ma anche la sua variabilita'.

Questa versione e' piu' ricca e piu' vicina ai problemi reali.

4. Cournot con costi eterogenei

4.1 Costi diversi tra imprese

Nel modello classico piu' semplice le imprese erano simmetriche. Introduciamo ora costi marginali diversi:

$$c_1 \neq c_2.$$

Il profitto dell'impresa 1 diventa

$$\pi_1 = q_1(a - b(q_1 + q_2) - c_1),$$

e quello dell'impresa 2

$$\pi_2 = q_2(a - b(q_1 + q_2) - c_2).$$

Le condizioni del primo ordine sono

$$a - c_1 - 2bq_1 - bq_2 = 0,$$

$$a - c_2 - 2bq_2 - bq_1 = 0.$$

4.2 Interpretazione

Questa estensione e' molto importante, perche' mostra che differenze anche semplici nei costi si traducono in:

- quantita' di equilibrio diverse;
- profitti diversi;
- maggiore o minore robustezza agli shock.

Dal punto di vista del corso, l'eterogeneita' e' uno dei modi piu' semplici per passare da un modello elegante ma uniforme a un modello piu' realistico.

5. Cournot con apprendimento adattivo

5.1 Perche' introdurre apprendimento

Nel modello classico ogni impresa conosce perfettamente la struttura del gioco e sceglie subito la best response. Questo e' utile teoricamente, ma poco plausibile in molti contesti.

Una descrizione piu' realistica e' la seguente:

- l'impresa osserva il mercato;

- aggiorna la propria scelta in modo graduale;
- non raggiunge necessariamente la best response in un solo passo.

5.2 Dinamica adattiva

Sia

$$R_1(q_2)$$

la funzione di reazione dell'impresa 1 e

$$R_2(q_1)$$

quella dell'impresa 2.

Una dinamica adattiva semplice e'

$$q_1(t+1) = q_1(t) + \lambda(R_1(q_2(t)) - q_1(t)),$$

$$q_2(t+1) = q_2(t) + \lambda(R_2(q_1(t)) - q_2(t)),$$

dove

$$0 < \lambda \leq 1$$

e' la velocita' di aggiustamento.

Interpretazione:

- se $\lambda = 1$, ogni impresa salta direttamente alla best response;
- se $\lambda < 1$, l'aggiustamento e' parziale e graduale.

5.3 Significato economico

Il parametro λ puo' rappresentare:

- inerzia decisionale;
- costi di aggiustamento;
- razionalita' limitata;
- apprendimento progressivo;
- lentezza organizzativa.

Questa e' una dinamica semplice ma molto utile, perche' trasforma il problema di equilibrio in un problema di traiettoria nel tempo.

6. Cournot con rumore decisionale

6.1 Aggiungere perturbazioni alla scelta

Si puo' fare un passo ulteriore introducendo rumore direttamente nella dinamica adattiva:

$$q_1(t+1) = q_1(t) + \lambda(R_1(q_2(t)) - q_1(t)) + \sigma\eta_1(t),$$

$$q_2(t+1) = q_2(t) + \lambda(R_2(q_1(t)) - q_2(t)) + \sigma\eta_2(t),$$

dove:

- $\eta_1(t), \eta_2(t)$ sono variabili casuali a media nulla;
- σ controlla l'intensita' del rumore.

6.2 Interpretazione

Il rumore decisionale puo' rappresentare:

- errori di previsione;
- aggiustamenti imperfetti;
- shock interni all'impresa;
- sperimentazione;
- bounded rationality.

A questo punto il sistema non converge piu' a un unico punto in modo perfetto, ma oscilla in un intorno dell'equilibrio, oppure puo' anche mostrare traiettorie molto piu' irregolari.

7. Hotelling con scelta probabilistica dei consumatori

7.1 Limite della versione classica

Nel modello classico di Hotelling ogni consumatore sceglie sempre il fornitore che minimizza il costo totale. Questa regola e' molto netta, ma spesso troppo rigida.

Una versione piu' realistica consiste nel rendere la scelta dei consumatori probabilistica.

7.2 Utilita' deterministica e componente casuale

Supponiamo che l'utilita' del consumatore x per acquistare dall'impresa 1 sia

$$U_1(x) = V - P_1 - t|x - x_1| + \xi_1,$$

e quella per l'impresa 2

$$U_2(x) = V - P_2 - t|x - x_2| + \xi_2,$$

dove:

- V e' una componente comune;
- P_1, P_2 sono i prezzi;
- t e' il costo di trasporto;
- ξ_1, ξ_2 sono componenti casuali individuali.

Il consumatore sceglie l'impresa con utilita' maggiore.

7.3 Effetto della scelta rumorosa

In questo caso il mercato non si divide piu' in modo netto nel solo punto indifferente. Si ottiene invece una quota di mercato probabilistica.

Questo e' molto importante:

- rende piu' realistico il comportamento dei consumatori;
- evita salti troppo bruschi nella domanda;
- rende il profitto una funzione piu' liscia delle scelte di prezzo e localizzazione.

Dal punto di vista computazionale, questa estensione si presta molto bene a simulazioni agent-based.

8. Hotelling con localizzazione perturbata

8.1 Localizzazione come scelta non perfetta

Anche la posizione delle imprese puo' essere trattata in modo adattivo e rumoroso.

Se chiamiamo

$$x_1(t), \quad x_2(t)$$

le localizzazioni al tempo t , una dinamica semplice e'

$$x_1(t+1) = x_1(t) + \gamma F_1(x_1(t), x_2(t)) + \sigma \zeta_1(t),$$

$$x_2(t+1) = x_2(t) + \gamma F_2(x_1(t), x_2(t)) + \sigma \zeta_2(t),$$

dove:

- F_1, F_2 rappresentano la direzione di miglior risposta;
- γ e' la velocita' di aggiustamento;
- ζ_1, ζ_2 sono shock casuali.

8.2 Interpretazione

Questa dinamica descrive bene contesti in cui:

- le imprese sperimentano posizionamenti diversi;
- non conoscono con precisione la mappa della domanda;
- apprendono gradualmente dal mercato;
- possono commettere errori di posizionamento.

9. Bounded rationality e apprendimento locale

9.1 Idea generale

Una seconda famiglia di modelli stocastici non assume che l'impresa conosca la best response, neanche in forma approssimata. L'impresa può invece usare regole locali molto semplici, per esempio:

- aumentare leggermente la quantità se il profitto è cresciuto;
- ridurla se il profitto è diminuito;
- imitare l'impresa più profittevole;
- sperimentare piccole variazioni casuali.

9.2 Esempio molto semplice

Sia

$$\Delta_i(t) = \pi_i(t) - \pi_i(t-1).$$

Una regola di aggiornamento minimale può essere:

$$q_i(t+1) = \begin{cases} q_i(t) + \delta & \text{se } \Delta_i(t) > 0, \\ q_i(t) - \delta & \text{se } \Delta_i(t) < 0, \end{cases}$$

con una piccola perturbazione casuale opzionale.

Questa regola è grezza, ma didatticamente molto interessante. Mostra che una dinamica di apprendimento non richiede ottimizzazione perfetta. Basta una regola adattiva locale.

10. Equilibrio contro distribuzione degli esiti

Nel passaggio dal modello deterministico al modello stocastico cambia anche l'oggetto di studio.

Nel modello deterministico si cerca:

- un punto di equilibrio;
- una funzione di reazione;

- una configurazione stabile.

Nel modello stocastico, invece, spesso interessa:

- la distribuzione dei prezzi;
- la distribuzione delle quantità;
- la volatilità dei profitti;
- il tempo medio di convergenza;
- la probabilità di grandi scostamenti;
- la sensibilità al rumore.

Questo è un passaggio metodologico cruciale per il corso.

11. Osservabili da misurare

Per trasformare questi modelli in case study computazionali conviene definire alcune osservabili.

11.1 Quantità medie

$$\bar{q}_i = \frac{1}{T} \sum_{t=1}^T q_i(t).$$

11.2 Prezzo medio

$$\bar{p} = \frac{1}{T} \sum_{t=1}^T p(t).$$

11.3 Profitto medio

$$\bar{\pi}_i = \frac{1}{T} \sum_{t=1}^T \pi_i(t).$$

11.4 Volatilità

$$\text{Var}(q_i), \quad \text{Var}(p), \quad \text{Var}(\pi_i).$$

11.5 Tempo di assestamento

Numero di passi necessari per entrare in un intorno di equilibrio o in una regione statisticamente stabile.

Queste osservabili permettono di confrontare direttamente il modello classico con le versioni rumorose e adattive.

12. Schema del laboratorio: Cournot stocastico

12.1 Laboratorio 1 - Domanda rumorosa

Obiettivo

Studiare come cambia la distribuzione dei profitti quando il prezzo contiene uno shock casuale.

Attivita'

1. fissare a , b , q_1 , q_2 ;
2. generare molti shock ε_t ;
3. calcolare i profitti realizzati;
4. confrontare media e varianza con il caso deterministico.

Domande guida

- il profitto medio cambia?
- quanto aumenta la dispersione dei profitti?
- quali imprese sono piu' vulnerabili agli shock?

12.2 Laboratorio 2 - Apprendimento adattivo

Obiettivo

Studiare la convergenza verso l'equilibrio sotto aggiornamento graduale.

Attivita'

1. fissare le funzioni di reazione;
2. scegliere diverse velocita' di aggiustamento λ ;
3. simulare la dinamica;
4. confrontare le traiettorie.

Domande guida

- la convergenza e' sempre garantita?
- valori alti di λ accelerano sempre la convergenza?
- esistono oscillazioni persistenti?

12.3 Laboratorio 3 - Rumore decisionale

Obiettivo

Studiare come il rumore nelle decisioni modifica le traiettorie.

Attivita'

1. introdurre il termine $\sigma\eta_i(t)$;
2. variare σ ;
3. misurare media, varianza e distribuzione delle quantita';
4. confrontare il risultato con il caso senza rumore.

Domande guida

- esiste un intorno stocastico dell'equilibrio?
- la volatilita' cresce linearmente con σ ?
- il sistema puo' occasionalmente allontanarsi molto dall'equilibrio?

13. Schema del laboratorio: Hotelling stocastico

13.1 Laboratorio 4 - Consumatori con scelta probabilistica

Obiettivo

Sostituire la divisione netta del mercato con una scelta discreta rumorosa.

Attivita'

1. generare molti consumatori distribuiti nello spazio;
2. assegnare utilita' con componente casuale;
3. simulare la scelta tra i due venditori;
4. calcolare quote di mercato e profitti.

Domande guida

- la quota di mercato resta una funzione netta della posizione?
- il rumore attenua la competizione di prezzo?
- l'effetto della distanza resta dominante?

13.2 Laboratorio 5 - Localizzazione adattiva

Obiettivo

Simulare imprese che correggono gradualmente la localizzazione.

Attivita'

1. inizializzare due posizioni;
2. definire una regola adattiva;
3. aggiungere piccole perturbazioni casuali;
4. osservare se emerge una tendenza alla convergenza verso il centro.

Domande guida

- la minima differenziazione resta osservabile?
- il rumore puo' impedire il collasso al centro?
- esistono configurazioni quasi stabili ma non perfettamente coincidenti?

14. Pseudocodice: Cournot adattivo rumoroso

Consideriamo il caso a due imprese.

1. fissare a, b, λ, σ, T
2. inizializzare $q_1(0)$ e $q_2(0)$
3. per $t = 0, \dots, T - 1$:
 - calcolare

$$R_1(q_2(t)), \quad R_2(q_1(t))$$

- generare due shock

$$\eta_1(t), \eta_2(t)$$

- aggiornare

$$q_1(t+1) = q_1(t) + \lambda(R_1(q_2(t)) - q_1(t)) + \sigma\eta_1(t)$$

$$q_2(t+1) = q_2(t) + \lambda(R_2(q_1(t)) - q_2(t)) + \sigma\eta_2(t)$$

- imporre eventualmente

$$q_i(t+1) \geq 0$$

- calcolare prezzo e profitti

4. salvare le traiettorie
5. ripetere per molte simulazioni indipendenti

Questo e' uno dei modelli piu' semplici ma piu' utili dell'intera dispensa.

15. Pseudocodice: Hotelling con consumatori simulati

1. fissare le posizioni x_1, x_2 e i prezzi P_1, P_2
2. generare molti consumatori x distribuiti sull'intervallo di mercato
3. per ogni consumatore:
 - calcolare

$$U_1(x) = V - P_1 - t|x - x_1| + \xi_1$$

$$U_2(x) = V - P_2 - t|x - x_2| + \xi_2$$

- assegnare il consumatore all'impresa con utilita' maggiore

4. contare le quote di mercato
5. calcolare i profitti
6. ripetere l'esperimento molte volte

Questa struttura agent-based e' molto trasparente e rende bene il passaggio dalla geometria del modello classico a una simulazione probabilistica.

16. Perché questa dispensa è importante

Questa seconda dispensa ha una funzione precisa nel corso.

Primo, mostra che i modelli classici non devono essere abbandonati, ma estesi.

Secondo, permette di introdurre strumenti computazionali fondamentali:

- simulazione Monte Carlo;
- traiettorie stocastiche;
- distribuzioni empiriche;
- confronto tra media e volatilità;
- apprendimento adattivo.

Terzo, rende più realistico il linguaggio della concorrenza economica, avvicinandolo a mercati in cui gli attori non sono perfettamente razionali e i dati non sono perfettamente stabili.

17. Conclusione

L'introduzione di rumore, eterogeneità e apprendimento adattivo trasforma i modelli di Cournot e Hotelling da esercizi eleganti di equilibrio deterministico in veri casi di studio per un corso di metodi computazionali per modelli stocastici.

A questo punto l'oggetto di studio non è più solo "il" punto di equilibrio, ma anche:

- la traiettoria di avvicinamento;
- la dispersione degli esiti;
- la robustezza agli shock;
- l'effetto dell'eterogeneità;
- il ruolo della razionalità limitata.

È proprio questa trasformazione che rende i modelli classici ancora oggi estremamente utili.

18. Bibliografia minima

1. Cournot, A. A. (1838). *Recherches sur les principes mathématiques de la théorie des richesses*.
2. Hotelling, H. (1929). *Stability in Competition*. *Economic Journal*, 39(153), 41-57.
3. Tirole, J. (1988). *The Theory of Industrial Organization*. MIT Press.
4. Fudenberg, D., and Levine, D. K. (1998). *The Theory of Learning in Games*. MIT Press.
5. Vega-Redondo, F. (2003). *Economics and the Theory of Games*. Cambridge University Press.

6.

Tesfatsion, L., and Judd, K. L. (eds.) (2006). Handbook of Computational Economics, Volume 2: Agent-Based Computational Economics. Elsevier.

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python i due modelli discussi nella dispensa:

1. il modello di Cournot;
2. il modello di Hotelling.

L'obiettivo non e' costruire un programma sofisticato, ma offrire uno schema chiaro che possa essere letto:

- come pseudocodice da chi usa altri linguaggi;
- come base quasi immediatamente eseguibile da chi conosce Python.

Per questo motivo il codice e' volutamente elementare:

- poche librerie;
- funzioni corte;
- cicli ed espressioni esplicite;
- nomi leggibili.

A.1 Librerie minime

Per questa appendice bastano:

```
import matplotlib.pyplot as plt
```

Se si vogliono usare funzioni matematiche elementari, si puo' aggiungere:

```
import math
```

In realta', per quasi tutto quello che segue, `math` non e' indispensabile.

Non e' necessario usare `numpy` in una prima implementazione.

A.2 Idea generale

La struttura del codice puo' essere organizzata in due blocchi.

Blocco 1 - Cournot

- definizione della domanda inversa;
- definizione del profitto;
- definizione delle funzioni di reazione;

- calcolo dell'equilibrio;
- grafico delle curve di reazione.

Blocco 2 - Hotelling

- dati i parametri di localizzazione e costo di trasporto;
- calcolo del consumatore indifferente;
- calcolo delle quantita' vendute;
- calcolo dei prezzi di equilibrio;
- eventuale grafico di comparativa statica.

A.3 Cournot: caso lineare con due imprese

Partiamo dal caso piu' semplice e piu' utile didatticamente.

La domanda inversa e'

$$p = a - b(D_1 + D_2),$$

con

$$a > 0, \quad b > 0.$$

Il profitto dell'impresa 1 e'

$$\pi_1 = D_1(a - b(D_1 + D_2)),$$

e analogamente per l'impresa 2.

A.4 Funzione prezzo

```
def price(a, b, q1, q2):
    p = a - b * (q1 + q2)
    return p
```

Qui:

- a e b sono i parametri della domanda;
- q1 e q2 sono le quantita' delle due imprese.

A.5 Profitti

```
def profit_firm_1(a, b, q1, q2):
    p = price(a, b, q1, q2)
    return q1 * p
```

```
def profit_firm_2(a, b, q1, q2):
    p = price(a, b, q1, q2)
    return q2 * p
```

Queste funzioni sono utili anche per fare controlli numerici o grafici del profitto.

A.6 Funzioni di reazione

Nel caso lineare, le funzioni di reazione sono esplicite:

$$q_1 = \frac{a - bq_2}{2b}, \quad q_2 = \frac{a - bq_1}{2b}.$$

In Python:

```
def reaction_1(a, b, q2):
    q1 = (a - b * q2) / (2 * b)
    if q1 < 0:
        q1 = 0
    return q1

def reaction_2(a, b, q1):
    q2 = (a - b * q1) / (2 * b)
    if q2 < 0:
        q2 = 0
    return q2
```

Il controllo `if q1 < 0` e `if q2 < 0` serve solo a evitare quantita' negative.

A.7 Equilibrio di Cournot in forma chiusa

Nel caso simmetrico con due imprese l'equilibrio e'

$$q_1^* = q_2^* = \frac{a}{3b}.$$

Possiamo quindi scrivere una funzione diretta:

```
def cournot_equilibrium_closed_form(a, b):
    q_star = a / (3 * b)
    p_star = price(a, b, q_star, q_star)

    results = {
        "q1_star": q_star,
        "q2_star": q_star,
        "p_star": p_star
    }

    return results
```

Esempio:

```
results = cournot_equilibrium_closed_form(a=12, b=1)

print("q1* =", results["q1_star"])
print("q2* =", results["q2_star"])
print("p*  =", results["p_star"])
```

A.8 Equilibrio di Cournot con iterazione

Anche se qui la soluzione chiusa esiste, e' molto utile didatticamente calcolare l'equilibrio con un'iterazione di best response. Questo prepara bene alle versioni piu' generali o stocastiche.

```
def cournot_best_response_iteration(a, b, q1_0, q2_0,
                                   tolerance=1e-8,
                                   max_steps=1000):

    q1 = q1_0
    q2 = q2_0

    history_q1 = [q1]
    history_q2 = [q2]

    for step in range(max_steps):
        q1_new = reaction_1(a, b, q2)
        q2_new = reaction_2(a, b, q1)

        history_q1.append(q1_new)
        history_q2.append(q2_new)

        if abs(q1_new - q1) < tolerance and abs(q2_new - q2) < tolerance:
            break

    q1 = q1_new
    q2 = q2_new

    p_star = price(a, b, q1_new, q2_new)

    results = {
        "q1_star": q1_new,
        "q2_star": q2_new,
        "p_star": p_star,
        "history_q1": history_q1,
        "history_q2": history_q2
    }

    return results
```

Esempio:

```
results = cournot_best_response_iteration(  
    a=12,  
    b=1,  
    q1_0=1,  
    q2_0=8  
)  
  
print("q1* =", results["q1_star"])  
print("q2* =", results["q2_star"])  
print("p*  =", results["p_star"])
```

A.9 Grafico delle curve di reazione

Questo e' uno dei grafici piu' utili nella parte su Cournot.

```
def plot_cournot_reactions(a, b, q_max=15, num_points=200):  
    q1_values = []  
    q2_values = []  
    r1_values = []  
    r2_values = []  
  
    for n in range(num_points + 1):  
        q = q_max * n / num_points  
  
        q2_values.append(q)  
        r1_values.append(reaction_1(a, b, q))  
  
        q1_values.append(q)  
        r2_values.append(reaction_2(a, b, q))  
  
    plt.plot(q2_values, r1_values, label="reazione impresa 1")  
    plt.plot(r2_values, q1_values, label="reazione impresa 2")  
    plt.xlabel("q2")  
    plt.ylabel("q1")  
    plt.title("Curve di reazione di Cournot")  
    plt.legend()  
    plt.show()
```

Esempio:

```
plot_cournot_reactions(a=12, b=1, q_max=15)
```

Nota: il secondo grafico e' scritto in modo da rappresentare entrambe le curve nello stesso piano (q_2, q_1).

A.10 Grafico della traiettoria verso l'equilibrio

Se si vuole vedere come la dinamica iterativa converge:

```
def plot_cournot_histories(history_q1, history_q2):
    times = list(range(len(history_q1)))

    plt.plot(times, history_q1, label="q1")
    plt.plot(times, history_q2, label="q2")
    plt.xlabel("iterazione")
    plt.ylabel("quantita'")
    plt.title("Convergenza verso l'equilibrio di Cournot")
    plt.legend()
    plt.show()
```

Esempio:

```
results = cournot_best_response_iteration(a=12, b=1, q1_0=1, q2_0=8)
plot_cournot_histories(results["history_q1"], results["history_q2"])
```

A.11 Generalizzazione a n imprese simmetriche

Se si vuole usare direttamente la formula simmetrica con n imprese, si può scrivere:

$$D^* = \frac{na}{(n+1)b}, \quad D_i^* = \frac{a}{(n+1)b}, \quad p^* = \frac{a}{n+1}.$$

In Python:

```
def cournot_symmetric_n_firms(a, b, n):
    total_quantity = n * a / ((n + 1) * b)
    individual_quantity = a / ((n + 1) * b)
    equilibrium_price = a / (n + 1)

    results = {
        "total_quantity": total_quantity,
        "individual_quantity": individual_quantity,
        "equilibrium_price": equilibrium_price
    }

    return results
```

Esempio:

```
for n in [1, 2, 3, 5, 10]:
    results = cournot_symmetric_n_firms(a=12, b=1, n=n)
    print(n, results)
```

Questo blocco e' molto utile per far vedere rapidamente come il prezzo scende quando cresce il numero di imprese.

A.12 Hotelling: struttura di base

Passiamo ora a Hotelling nella versione della dispensa.

I parametri sono:

- lunghezza del mercato ℓ ;
- posizione della prima impresa a ;
- posizione della seconda impresa b rispetto all'altro estremo;
- costo di trasporto per unita' di distanza c .

L'equilibrio dei prezzi, nella forma riportata nella dispensa, e':

$$P_1^* = c\ell + \frac{c}{3}(a - b),$$

$$P_2^* = c\ell - \frac{c}{3}(a - b).$$

Le quantita' corrispondenti sono:

$$q_1^* = \frac{1}{2} \left(\ell + \frac{a - b}{3} \right), \quad q_2^* = \frac{1}{2} \left(\ell - \frac{a - b}{3} \right).$$

A.13 Funzione per l'equilibrio di Hotelling

```
def hotelling_price_equilibrium(ell, a, b, c):  
    p1_star = c * ell + (c / 3) * (a - b)  
    p2_star = c * ell - (c / 3) * (a - b)  
  
    q1_star = 0.5 * (ell + (a - b) / 3)  
    q2_star = 0.5 * (ell - (a - b) / 3)  
  
    results = {  
        "p1_star": p1_star,  
        "p2_star": p2_star,  
        "q1_star": q1_star,  
        "q2_star": q2_star  
    }  
  
    return results
```

Esempio:

```

results = hotelling_price_equilibrium(
    ell=10,
    a=2,
    b=1,
    c=1
)

print("P1* =", results["p1_star"])
print("P2* =", results["p2_star"])
print("q1* =", results["q1_star"])
print("q2* =", results["q2_star"])

```

A.14 Consumatori indifferenti e domanda intermedia

Se si vuole calcolare anche il consumatore indifferente nelle formule intermedie, si può usare:

```

def hotelling_indifferent_consumer(ell, a, b, c, p1, p2):
    x = 0.5 * (ell - a - b) + 0.5 * (p2 - p1) / c
    y = 0.5 * (ell - a - b) - 0.5 * (p2 - p1) / c

    results = {
        "x": x,
        "y": y
    }

    return results

```

Poi:

```

def hotelling_quantities(ell, a, b, c, p1, p2):
    middle = hotelling_indifferent_consumer(ell, a, b, c, p1, p2)

    x = middle["x"]
    y = middle["y"]

    q1 = a + x
    q2 = b + y

    results = {
        "q1": q1,
        "q2": q2
    }

    return results

```

Queste funzioni sono utili se si vuole distinguere tra:

- calcolo generale delle quantità;
- formula già risolta in equilibrio.

A.15 Profitti in Hotelling

```
def hotelling_profits(ell, a, b, c, p1, p2):
    quantities = hotelling_quantities(ell, a, b, c, p1, p2)

    q1 = quantities["q1"]
    q2 = quantities["q2"]

    profit_1 = p1 * q1
    profit_2 = p2 * q2

    results = {
        "profit_1": profit_1,
        "profit_2": profit_2,
        "q1": q1,
        "q2": q2
    }

    return results
```

Esempio:

```
results = hotelling_profits(
    ell=10,
    a=2,
    b=1,
    c=1,
    p1=10,
    p2=9
)

print(results)
```

A.16 Comparativa statica rispetto alla localizzazione

Una cosa molto utile da mostrare agli studenti è come cambino prezzi e quantità di equilibrio al variare di $a - b$.

```
def hotelling_statics_over_a(ell, a_values, b, c):
    p1_values = []
    p2_values = []
    q1_values = []
    q2_values = []
```

```

for a in a_values:
    results = hotelling_price_equilibrium(ell, a, b, c)

    p1_values.append(results["p1_star"])
    p2_values.append(results["p2_star"])
    q1_values.append(results["q1_star"])
    q2_values.append(results["q2_star"])

summary = {
    "p1_values": p1_values,
    "p2_values": p2_values,
    "q1_values": q1_values,
    "q2_values": q2_values
}

return summary

```

Grafico dei prezzi:

```

def plot_hotelling_prices(a_values, p1_values, p2_values):
    plt.plot(a_values, p1_values, label="P1*")
    plt.plot(a_values, p2_values, label="P2*")
    plt.xlabel("a")
    plt.ylabel("prezzi di equilibrio")
    plt.title("Hotelling: prezzi di equilibrio")
    plt.legend()
    plt.show()

```

Grafico delle quantita':

```

def plot_hotelling_quantities(a_values, q1_values, q2_values):
    plt.plot(a_values, q1_values, label="q1*")
    plt.plot(a_values, q2_values, label="q2*")
    plt.xlabel("a")
    plt.ylabel("quantita' di equilibrio")
    plt.title("Hotelling: quote di mercato")
    plt.legend()
    plt.show()

```

Esempio completo:

```

a_values = [0.5 + 0.1 * n for n in range(31)]

summary = hotelling_statics_over_a(
    ell=10,
    a_values=a_values,
    b=1,
    c=1
)

```

```

plot_hotelling_prices(a_values, summary["p1_values"], summary["p2_values"])
plot_hotelling_quantities(a_values, summary["q1_values"], summary["q2_values"])

```

A.17 Organizzazione consigliata del file

Per mantenere il codice leggibile, conviene organizzarlo così:

1. import delle librerie;
2. funzioni per Cournot:
 - price
 - profit_firm_1
 - profit_firm_2
 - reaction_1
 - reaction_2
 - cournot_equilibrium_closed_form
 - cournot_best_response_iteration
 - funzioni di grafico
3. funzioni per Hotelling:
 - hotelling_indifferent_consumer
 - hotelling_quantities
 - hotelling_profits
 - hotelling_price_equilibrium
 - funzioni di comparativa statica
4. blocco finale con esempi.

Per esempio:

```

if __name__ == "__main__":
    results_c = cournot_best_response_iteration(
        a=12,
        b=1,
        q1_0=1,
        q2_0=8
    )

    print("Cournot:")
    print("q1* =", results_c["q1_star"])
    print("q2* =", results_c["q2_star"])
    print("p*  =", results_c["p_star"])

    plot_cournot_reactions(a=12, b=1, q_max=15)
    plot_cournot_histories(results_c["history_q1"], results_c["history_q2"])

    results_h = hotelling_price_equilibrium(

```

```

    ell=10,
    a=2,
    b=1,
    c=1
)

print("Hotelling:")
print(results_h)

```

A.18 Perché questa appendice è utile

Questa appendice ha due funzioni didattiche.

Primo, mostra che i modelli classici della concorrenza possono essere tradotti in codice con grande semplicità. Non serve nessuna tecnica avanzata per implementare:

- funzioni di reazione;
- equilibri chiusi;
- iterazioni;
- grafici di comparativa statica.

Secondo, prepara naturalmente il passaggio alla seconda dispensa, dove questi stessi modelli saranno modificati introducendo:

- rumore;
- eterogeneità;
- apprendimento adattivo;
- bounded rationality.

In quel momento il codice qui costruito funzionerà come base da estendere, e non come esercizio isolato.

A.19 Conclusione dell'appendice

La struttura proposta qui è volutamente semplice. Chi usa Python può implementarla quasi direttamente; chi usa altri linguaggi può leggerla come pseudocodice molto vicino a una traduzione operativa.

Il messaggio metodologico è importante: anche modelli teorici classici come Cournot e Hotelling possono essere trasformati subito in oggetti computazionali. Questo rende più trasparente la logica dell'equilibrio e prepara bene il terreno per modelli più ricchi e più realistici.

Appendice B. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python i modelli stocastici discussi nella dispensa:

1. Cournot con rumore, eterogeneità e apprendimento adattivo;
2. Hotelling con scelta probabilistica dei consumatori e, in forma elementare, localizzazione adattiva.

L'obiettivo non è costruire un programma sofisticato, ma fornire una guida leggibile anche da chi usa altri linguaggi di programmazione. Per questo motivo il codice è volutamente semplice:

- poche librerie;
- funzioni corte;
- passaggi espliciti;
- nomi leggibili;
- strutture facilmente traducibili in altri linguaggi.

L'idea generale è sempre la stessa:

1. definire lo stato del sistema;
2. scrivere le regole di aggiornamento;
3. simulare la dinamica;
4. salvare le traiettorie;
5. analizzare media, volatilità e distribuzioni finali.

B.1 Librerie minime

Per questa appendice bastano poche librerie standard.

```
import random
import statistics
import matplotlib.pyplot as plt
```

Quindi:

- `random` serve per generare shock e scelte casuali;
- `statistics` serve per media e deviazione standard;
- `matplotlib.pyplot` serve per i grafici.

Non è necessario usare `numpy` in una prima implementazione.

B.2 Parte I - Cournot stocastico

B.2.1 Domanda lineare con shock

Nel modello stocastico di Cournot, il prezzo può essere scritto come

$$p(t) = a - b(q_1(t) + q_2(t)) + \varepsilon_t.$$

Qui:

- a è l'intercetta della domanda;
- b è la pendenza;
- q_1, q_2 sono le quantità;
- ε_t è uno shock casuale.

La funzione prezzo può essere scritta così:

```
def cournot_price(a, b, q1, q2, epsilon=0.0):
    p = a - b * (q1 + q2) + epsilon
    return p
```

B.2.2 Profitti con costi eterogenei

Se le imprese hanno costi marginali diversi c_1 e c_2 , i profitti sono

$$\pi_1 = q_1(p - c_1), \quad \pi_2 = q_2(p - c_2).$$

In Python:

```
def cournot_profit_1(a, b, q1, q2, c1, epsilon=0.0):
    p = cournot_price(a, b, q1, q2, epsilon=epsilon)
    return q1 * (p - c1)

def cournot_profit_2(a, b, q1, q2, c2, epsilon=0.0):
    p = cournot_price(a, b, q1, q2, epsilon=epsilon)
    return q2 * (p - c2)
```

B.2.3 Funzioni di reazione con costi diversi

Con domanda lineare e costi marginali diversi, le funzioni di reazione sono

$$R_1(q_2) = \frac{a - c_1 - bq_2}{2b}, \quad R_2(q_1) = \frac{a - c_2 - bq_1}{2b}.$$

In Python:

```
def cournot_reaction_1(a, b, c1, q2):
    q1 = (a - c1 - b * q2) / (2 * b)
    if q1 < 0:
        q1 = 0.0
    return q1

def cournot_reaction_2(a, b, c2, q1):
```

```

q2 = (a - c2 - b * q1) / (2 * b)
if q2 < 0:
    q2 = 0.0
return q2

```

B.2.4 Dinamica adattiva senza rumore

Una dinamica adattiva elementare è

$$q_i(t+1) = q_i(t) + \lambda(R_i - q_i(t)).$$

Qui λ è la velocità di aggiustamento. In Python:

```

def cournot_update_adaptive(q1, q2, a, b, c1, c2, lambd):
    r1 = cournot_reaction_1(a, b, c1, q2)
    r2 = cournot_reaction_2(a, b, c2, q1)

    q1_new = q1 + lambd * (r1 - q1)
    q2_new = q2 + lambd * (r2 - q2)

    if q1_new < 0:
        q1_new = 0.0
    if q2_new < 0:
        q2_new = 0.0

    return q1_new, q2_new

```

B.2.5 Dinamica adattiva con rumore decisionale

Per introdurre rumore, si aggiunge un termine casuale alle quantità:

$$q_i(t+1) = q_i(t) + \lambda(R_i - q_i(t)) + \sigma\eta_i(t).$$

In Python:

```

def cournot_update_noisy(q1, q2, a, b, c1, c2, lambd, sigma):
    r1 = cournot_reaction_1(a, b, c1, q2)
    r2 = cournot_reaction_2(a, b, c2, q1)

    noise_1 = random.gauss(0.0, 1.0)
    noise_2 = random.gauss(0.0, 1.0)

    q1_new = q1 + lambd * (r1 - q1) + sigma * noise_1
    q2_new = q2 + lambd * (r2 - q2) + sigma * noise_2

    if q1_new < 0:

```

```

        q1_new = 0.0
    if q2_new < 0:
        q2_new = 0.0

    return q1_new, q2_new

```

Qui si usa `random.gauss(0.0, 1.0)` per generare un rumore normale standard.

B.2.6 Una simulazione completa di Cournot

Conviene ora scrivere una funzione che simuli l'intera traiettoria temporale.

```

def run_cournot_simulation(a, b, c1, c2, lambd, sigma,
                           q1_0, q2_0, T, demand_shock_sigma=0.0):
    q1 = q1_0
    q2 = q2_0

    history_q1 = [q1]
    history_q2 = [q2]
    history_price = []
    history_profit_1 = []
    history_profit_2 = []

    for t in range(T):
        epsilon = random.gauss(0.0, demand_shock_sigma)

        q1, q2 = cournot_update_noisy(
            q1=q1,
            q2=q2,
            a=a,
            b=b,
            c1=c1,
            c2=c2,
            lambd=lambd,
            sigma=sigma
        )

        p = cournot_price(a, b, q1, q2, epsilon=epsilon)
        pi1 = cournot_profit_1(a, b, q1, q2, c1, epsilon=epsilon)
        pi2 = cournot_profit_2(a, b, q1, q2, c2, epsilon=epsilon)

        history_q1.append(q1)
        history_q2.append(q2)
        history_price.append(p)
        history_profit_1.append(pi1)
        history_profit_2.append(pi2)

```



```

results = {
    "history_q1": history_q1,
    "history_q2": history_q2,
    "history_price": history_price,
    "history_profit_1": history_profit_1,
    "history_profit_2": history_profit_2
}

return results

```

Nota importante: qui ci sono due tipi distinti di rumore:

- `sigma` controlla il rumore nella decisione delle imprese;
- `demand_shock_sigma` controlla il rumore nella domanda.

Questa distinzione è concettualmente utile e didatticamente molto chiara.

B.2.7 Esempio minimo

```

results = run_cournot_simulation(
    a=12.0,
    b=1.0,
    c1=2.0,
    c2=3.0,
    lambd=0.4,
    sigma=0.1,
    q1_0=1.0,
    q2_0=8.0,
    T=100,
    demand_shock_sigma=0.3
)

print("q1 finale =", results["history_q1"][-1])
print("q2 finale =", results["history_q2"][-1])

```

B.2.8 Grafici delle traiettorie

```

def plot_cournot_quantities(history_q1, history_q2):
    times = list(range(len(history_q1)))

    plt.plot(times, history_q1, label="q1")
    plt.plot(times, history_q2, label="q2")
    plt.xlabel("tempo")
    plt.ylabel("quantità")
    plt.title("Cournot stocastico: traiettorie delle quantità")
    plt.legend()
    plt.show()

```

```
def plot_cournot_profits(history_profit_1, history_profit_2):
    times = list(range(len(history_profit_1)))

    plt.plot(times, history_profit_1, label="profitto 1")
    plt.plot(times, history_profit_2, label="profitto 2")
    plt.xlabel("tempo")
    plt.ylabel("profitto")
    plt.title("Cournot stocastico: traiettorie dei profitti")
    plt.legend()
    plt.show()
```

Esempio:

```
plot_cournot_quantities(results["history_q1"], results["history_q2"])
plot_cournot_profits(results["history_profit_1"], results["history_profit_2"])
```

B.2.9 Molte simulazioni indipendenti

Per passare da una singola traiettoria a un'analisi Monte Carlo, conviene ripetere l'esperimento molte volte.

```
def run_many_cournot_simulations(num_runs, a, b, c1, c2, lambd, sigma,
                                q1_0, q2_0, T, demand_shock_sigma=0.0):

    final_q1 = []
    final_q2 = []
    final_p1 = []
    final_p2 = []

    for run in range(num_runs):
        results = run_cournot_simulation(
            a=a,
            b=b,
            c1=c1,
            c2=c2,
            lambd=lambd,
            sigma=sigma,
            q1_0=q1_0,
            q2_0=q2_0,
            T=T,
            demand_shock_sigma=demand_shock_sigma
        )

        final_q1.append(results["history_q1"][-1])
        final_q2.append(results["history_q2"][-1])
        final_p1.append(results["history_profit_1"][-1])
        final_p2.append(results["history_profit_2"][-1])
```

```

summary = {
    "mean_q1": statistics.mean(final_q1),
    "mean_q2": statistics.mean(final_q2),
    "std_q1": statistics.stdev(final_q1) if len(final_q1) > 1 else 0.0,
    "std_q2": statistics.stdev(final_q2) if len(final_q2) > 1 else 0.0,
    "mean_profit_1": statistics.mean(final_p1),
    "mean_profit_2": statistics.mean(final_p2)
}

return summary

```

Questo blocco è molto utile per costruire tabelle di comparativa statica rispetto a σ , λ , c_1 , c_2 .

B.3 Parte II - Hotelling con scelta probabilistica

B.3.1 Idee di base

Nella versione stocastica di Hotelling non si assume più che ogni consumatore scelga deterministicamente il venditore più vicino. Si assume invece che la scelta contenga una componente casuale.

Ogni consumatore confronta due utilità:

$$U_1(x) = V - P_1 - t|x - x_1| + \xi_1,$$

$$U_2(x) = V - P_2 - t|x - x_2| + \xi_2.$$

Per implementare una versione semplice, conviene simulare esplicitamente molti consumatori.

B.3.2 Generare i consumatori

Possiamo rappresentare il mercato come l'intervallo $[0, L]$. Ogni consumatore è identificato dalla sua posizione.

```

def create_consumers(num_consumers, L):
    consumers = []

    for n in range(num_consumers):
        x = random.uniform(0.0, L)
        consumers.append(x)

    return consumers

```

B.3.3 Utilità dei consumatori

```
def utility_firm_1(x, V, P1, x1, transport_cost, noise_sigma):
    noise = random.gauss(0.0, noise_sigma)
    return V - P1 - transport_cost * abs(x - x1) + noise

def utility_firm_2(x, V, P2, x2, transport_cost, noise_sigma):
    noise = random.gauss(0.0, noise_sigma)
    return V - P2 - transport_cost * abs(x - x2) + noise
```

Qui `noise_sigma` controlla il grado di rumorosità delle preferenze.

B.3.4 Assegnazione dei consumatori alle imprese

```
def assign_consumers(consumers, V, P1, P2, x1, x2, transport_cost, noise_sigma):
    demand_1 = 0
    demand_2 = 0

    for x in consumers:
        u1 = utility_firm_1(x, V, P1, x1, transport_cost, noise_sigma)
        u2 = utility_firm_2(x, V, P2, x2, transport_cost, noise_sigma)

        if u1 >= u2:
            demand_1 += 1
        else:
            demand_2 += 1

    return demand_1, demand_2
```

Questa è la forma più semplice e più didattica della scelta discreta rumorosa.

B.3.5 Profitti in Hotelling stocastico

```
def hotelling_profits_simulated(consumers, V, P1, P2, x1, x2,
                                transport_cost, noise_sigma):
    demand_1, demand_2 = assign_consumers(
        consumers=consumers,
        V=V,
        P1=P1,
        P2=P2,
        x1=x1,
        x2=x2,
        transport_cost=transport_cost,
        noise_sigma=noise_sigma
    )

    profit_1 = P1 * demand_1
```

```

profit_2 = P2 * demand_2

results = {
    "demand_1": demand_1,
    "demand_2": demand_2,
    "profit_1": profit_1,
    "profit_2": profit_2
}

return results

```

B.3.6 Esempio minimo di simulazione Hotelling

```

consumers = create_consumers(num_consumers=1000, L=10.0)

results = hotelling_profits_simulated(
    consumers=consumers,
    V=20.0,
    P1=5.0,
    P2=5.0,
    x1=3.0,
    x2=7.0,
    transport_cost=1.0,
    noise_sigma=0.5
)

print(results)

```

B.3.7 Ripetere molte simulazioni

Perche' il risultato non dipenda da una sola realizzazione casuale, conviene ripetere l'esperimento.

```

def run_many_hotelling_simulations(num_runs, num_consumers, L,
                                   V, P1, P2, x1, x2,
                                   transport_cost, noise_sigma):

    profits_1 = []
    profits_2 = []
    demands_1 = []
    demands_2 = []

    for run in range(num_runs):
        consumers = create_consumers(num_consumers, L)

        results = hotelling_profits_simulated(
            consumers=consumers,

```

```

        V=V,
        P1=P1,
        P2=P2,
        x1=x1,
        x2=x2,
        transport_cost=transport_cost,
        noise_sigma=noise_sigma
    )

    profits_1.append(results["profit_1"])
    profits_2.append(results["profit_2"])
    demands_1.append(results["demand_1"])
    demands_2.append(results["demand_2"])

summary = {
    "mean_profit_1": statistics.mean(profits_1),
    "mean_profit_2": statistics.mean(profits_2),
    "mean_demand_1": statistics.mean(demands_1),
    "mean_demand_2": statistics.mean(demands_2),
    "std_profit_1": statistics.stdev(profits_1) if len(profits_1) > 1 else 0.0,
    "std_profit_2": statistics.stdev(profits_2) if len(profits_2) > 1 else 0.0
}

return summary

```

B.3.8 Comparativa statica rispetto al rumore

Una delle esercitazioni piu' naturali è variare `noise_sigma` e osservare come cambiano quote di mercato e profitti medi.

```

def hotelling_noise_experiment(noise_values, num_runs, num_consumers, L,
                               V, P1, P2, x1, x2, transport_cost):
    mean_profit_1_values = []
    mean_profit_2_values = []

    for noise_sigma in noise_values:
        summary = run_many_hotelling_simulations(
            num_runs=num_runs,
            num_consumers=num_consumers,
            L=L,
            V=V,
            P1=P1,
            P2=P2,
            x1=x1,
            x2=x2,
            transport_cost=transport_cost,

```

```

        noise_sigma=noise_sigma
    )

    mean_profit_1_values.append(summary["mean_profit_1"])
    mean_profit_2_values.append(summary["mean_profit_2"])

    return mean_profit_1_values, mean_profit_2_values

```

Grafico:

```

def plot_hotelling_noise_experiment(noise_values, mean_profit_1_values, mean_profit_2_values):
    plt.plot(noise_values, mean_profit_1_values, label="profitto medio 1")
    plt.plot(noise_values, mean_profit_2_values, label="profitto medio 2")
    plt.xlabel("rumore delle preferenze")
    plt.ylabel("profitto medio")
    plt.title("Hotelling stocastico: effetto del rumore")
    plt.legend()
    plt.show()

```

B.4 Parte III - Hotelling con localizzazione adattiva elementare

B.4.1 Idea generale

Una versione ancora semplice consiste nel far muovere le imprese nello spazio in piccoli passi, osservando se il profitto migliora oppure no.

Questa non è una best response esatta. È una regola adattiva locale.

B.4.2 Un passo di aggiornamento locale

L'idea è:

- l'impresa prova a spostarsi leggermente a destra o a sinistra;
- confronta il profitto;
- mantiene lo spostamento se migliora il risultato.

Per la prima impresa:

```

def local_search_step_firm_1(consumers, V, P1, P2, x1, x2,
                             transport_cost, noise_sigma, delta, L):
    current_results = hotelling_profits_simulated(
        consumers, V, P1, P2, x1, x2, transport_cost, noise_sigma
    )
    current_profit = current_results["profit_1"]

    candidate_left = max(0.0, x1 - delta)
    candidate_right = min(L, x1 + delta)

```

```

results_left = hotelling_profits_simulated(
    consumers, V, P1, P2, candidate_left, x2, transport_cost, noise_sigma
)
results_right = hotelling_profits_simulated(
    consumers, V, P1, P2, candidate_right, x2, transport_cost, noise_sigma
)

best_x1 = x1
best_profit = current_profit

if results_left["profit_1"] > best_profit:
    best_profit = results_left["profit_1"]
    best_x1 = candidate_left

if results_right["profit_1"] > best_profit:
    best_profit = results_right["profit_1"]
    best_x1 = candidate_right

return best_x1

```

Una funzione analoga si scrive per la seconda impresa.

Questa struttura è molto semplice, ma didatticamente utile: mostra un apprendimento locale senza ottimizzazione completa.

B.5 Organizzazione consigliata del file

Per mantenere il codice leggibile, conviene organizzarlo in questo ordine:

1. import delle librerie;
2. blocco Cournot:
 - prezzo
 - profitti
 - funzioni di reazione
 - aggiornamento adattivo
 - simulazione completa
 - grafici
3. blocco Hotelling:
 - generazione dei consumatori
 - utilità
 - assegnazione
 - profitti simulati
 - molte simulazioni
 - eventuale aggiornamento locale delle posizioni
4. blocco finale con esempi.

Per esempio:

```
if __name__ == "__main__":
    cournot_results = run_cournot_simulation(
        a=12.0,
        b=1.0,
        c1=2.0,
        c2=3.0,
        lambd=0.4,
        sigma=0.1,
        q1_0=1.0,
        q2_0=8.0,
        T=100,
        demand_shock_sigma=0.3
    )

    plot_cournot_quantities(
        cournot_results["history_q1"],
        cournot_results["history_q2"]
    )

    consumers = create_consumers(num_consumers=1000, L=10.0)

    hotelling_results = hotelling_profits_simulated(
        consumers=consumers,
        V=20.0,
        P1=5.0,
        P2=5.0,
        x1=3.0,
        x2=7.0,
        transport_cost=1.0,
        noise_sigma=0.5
    )

    print(hotelling_results)
```

B.6 Perché questa appendice è utile

Questa appendice ha due vantaggi.

Primo, mostra che il passaggio dal modello teorico al modello computazionale non richiede strumenti avanzati. Basta scomporre il problema in pezzi chiari:

- stato;
- regole di aggiornamento;
- simulazione;
- osservabili;

- ripetizione Monte Carlo.

Secondo, la struttura del codice è leggibile sia come Python reale sia come pseudocodice facilmente traducibile in altri linguaggi.

B.7 Conclusione dell'appendice

I modelli di Cournot e Hotelling, una volta arricchiti con rumore, eterogeneità e apprendimento adattivo, diventano casi di studio molto adatti a un corso di metodi computazionali per modelli stocastici.

L'implementazione proposta qui è volutamente essenziale. Chi conosce Python può usarla quasi direttamente. Chi usa altri linguaggi può leggerla come una guida strutturata alla costruzione del modello.

Il punto metodologico importante è che, in questa seconda dispensa, l'oggetto di studio non è più soltanto un equilibrio puntuale, ma una dinamica stocastica fatta di traiettorie, distribuzioni, medie, volatilità e adattamento.